Evidencias de las pruebas mínimas

Requerimiento avanzado: cola de emergencias, asignación atómica, videollamada **WebRTC** con señalización por **Flask-SocketIO, STUN/TURN** y seguridad por filas en **PostgreSQL**.

Integrantes:	Daniel Gómez Urueta
	Fabian Muñoz Padilla

Fecha:	Noviembre 2026

Resumen

Prueba	Resultado esperado	Resultado
1. Socket.IO sin token o con token vencido	Conexión rechazada	Aprobada (se rechazaron las tres conexiones inválidas)
2. Dos operadores aceptan la misma emergencia a la vez	Solo uno la obtiene	Aprobada (14 de 14)
3. Un tercero envía webrtc:offer a una sesión ajena	No se retransmite	Aprobada (10 de 10)
4. Llamada celular con datos móviles ↔ PC en Wi-Fi	Conecta (STUN/TURN)	Aprobada
5. Tres emergencias en cola	Posiciones 1, 2, 3 y luego 1, 2	Aprobada
6. Consulta directa a la base como ciudadano	Solo ve sus sesiones	Aprobada

Prueba 1: Socket.IO sin token o con token vencido

Cómo se probó: script pruebas/probar_socket.py contra el servidor local. Intenta conectarse sin token, con un token falso, con un token vencido (firmado con la clave real pero con fecha de vencimiento en el pasado) y con un token válido, por polling y por websocket.
Resultado esperado: las conexiones sin token, con token falso y con token vencido se rechazan; la del token válido se acepta.

Resultado obtenido: se rechazaron las tres conexiones inválidas y se aceptó la válida, en los dos transportes. El servidor registró el motivo de cada rechazo (DecodeError y ExpiredSignatureError).

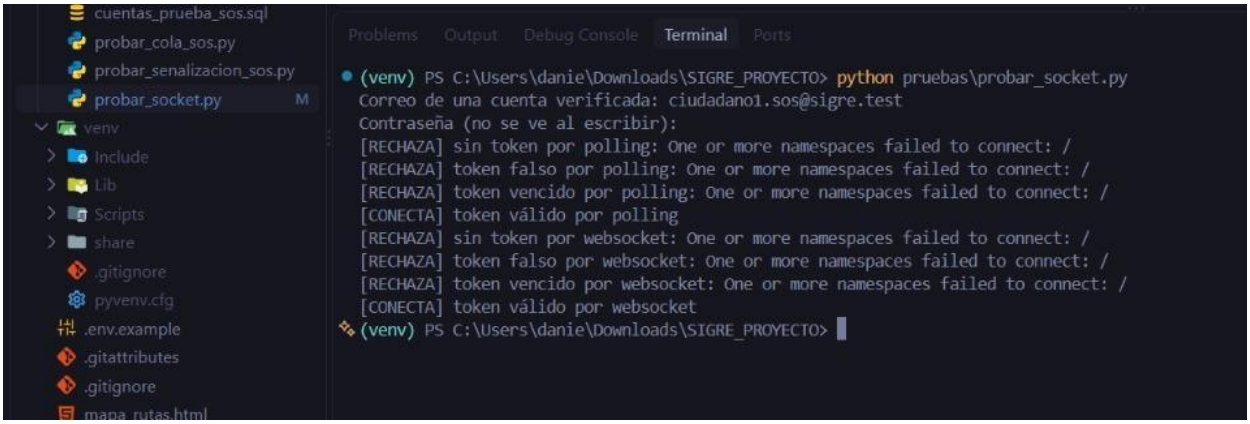


Figura 1. Resultado de probar_socket.py.



Figura 2. Registro del servidor: token falso (DecodeError) y token vencido (ExpiredSignatureError).

Prueba 2: dos operadores aceptan la misma emergencia al mismo tiempo

Cómo se probó: script pruebas/probar_cola_sos.py. Se reinician las cuentas de prueba (pruebas/cuentas_prueba_sos.sql) y dos operadores envían sos:aceptar a la vez sobre la misma emergencia.
Resultado esperado: solo un operador obtiene la emergencia; el otro recibe «ya fue tomada por otro operador».

Resultado obtenido: un operador recibió ok y el otro «Esta emergencia ya fue tomada por otro operador». La asignación es atómica (UPDATE condicionado en PostgreSQL). 14 de 14 verificaciones aprobadas.



Figura 3. Reinicio de las cuentas de prueba.

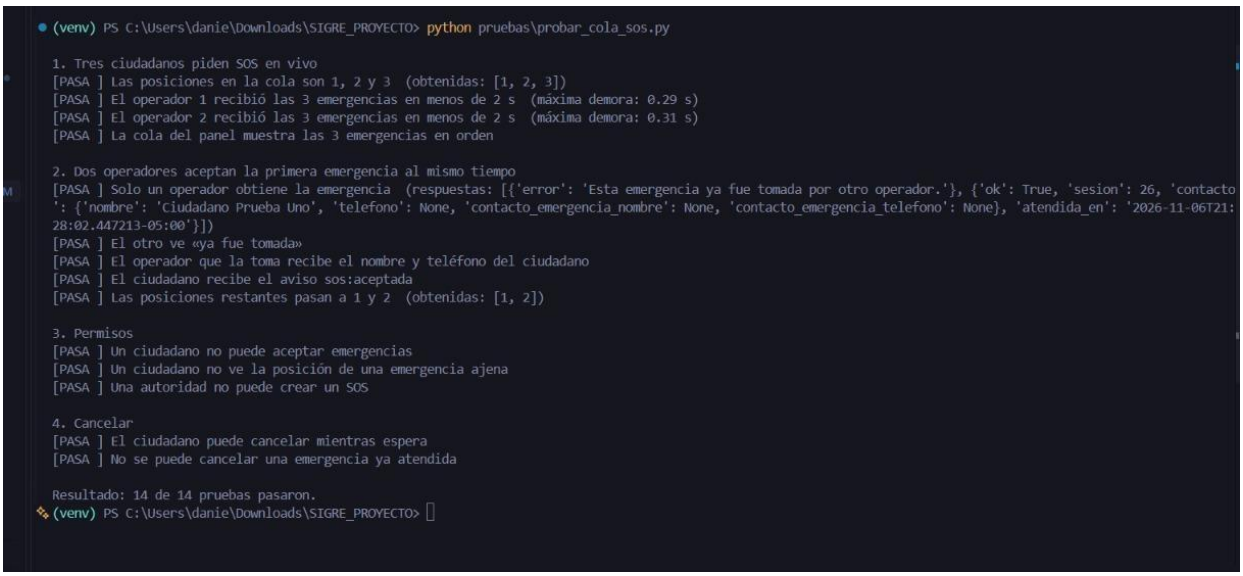


Figura 4. Resultado de probar_cola_sos.py (sección 2).

Prueba 3: un tercero envía webrtc:offer a una sesión ajena

Cómo se probó: script pruebas/probar_senalizacion_sos.py. Un ciudadano y un operador quedan en una llamada; otro operador y otro ciudadano intentan enviar señalización a esa emergencia.
Resultado esperado: el servidor rechaza la señalización de quien no participa y no la retransmite.

Resultado obtenido: el servidor respondió «No participas en esta emergencia» y el otro operador no recibió ningún mensaje de la llamada. 10 de 10 verificaciones aprobadas.

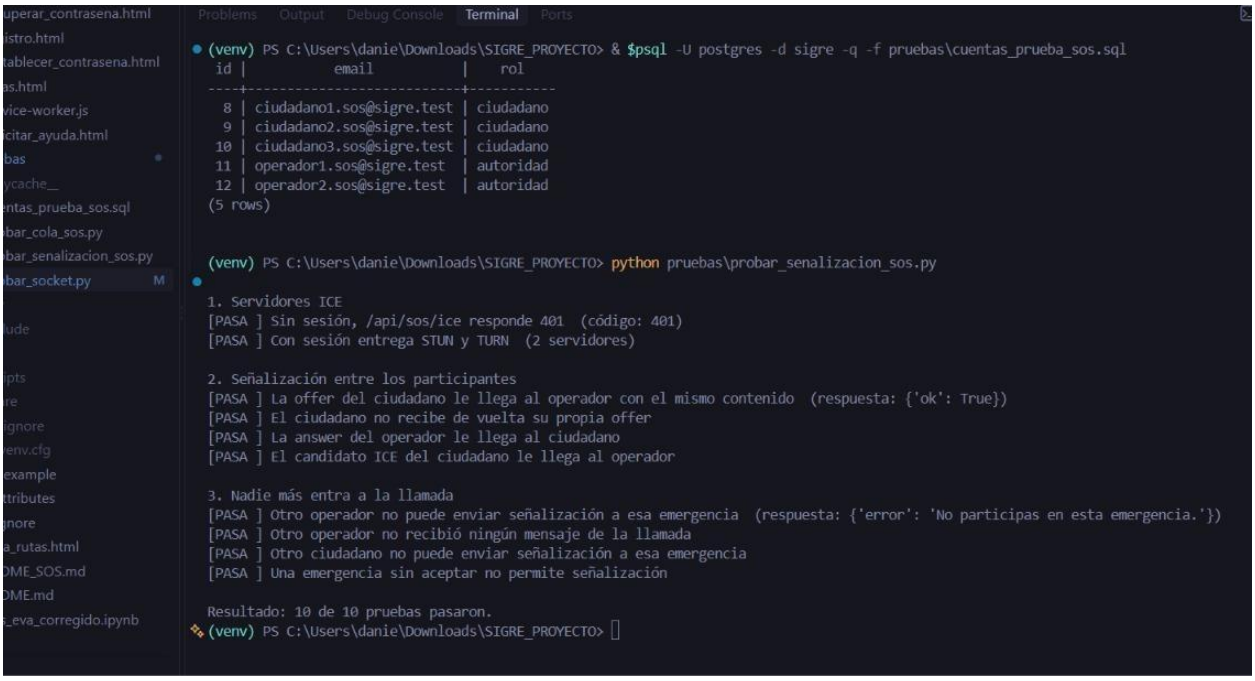


Figura 5. Resultado de probar_senalizacion_sos.py.

Prueba 4: llamada entre un celular con datos móviles y un computador en Wi-Fi

Cómo se probó: en https://sigre.onrender.com, el operador atiende desde un computador en Wi-Fi y el ciudadano pide SOS desde otro dispositivo con datos móviles. En chrome://webrtc-internals del computador del operador se revisa el par de candidatos usado.
Resultado esperado: la videollamada conecta entre las dos redes usando los servidores ICE (STUN y TURN de Metered), con video, audio y ubicación en vivo en el panel del operador.

Resultado obtenido: Aprobada. La videollamada conectó (ICE connection state: connected, par de candidatos succeeded, 85 MB de video recibidos). El computador del operador salió por Wi-Fi (IP pública 181.71.10.174) y el otro dispositivo por una red distinta (IP pública 191.156.95.143; IP privada 10.217.129.205, propia de datos móviles). Ambos candidatos fueron de tipo srflx (STUN): la conexión directa fue posible y no hizo falta retransmitir por el TURN, que quedó configurado en iceServers (turn:global.relay.metered.ca, puertos 80 y 443). En el panel se vieron el video del ciudadano, el cronómetro de la llamada y su ubicación en el mapa.

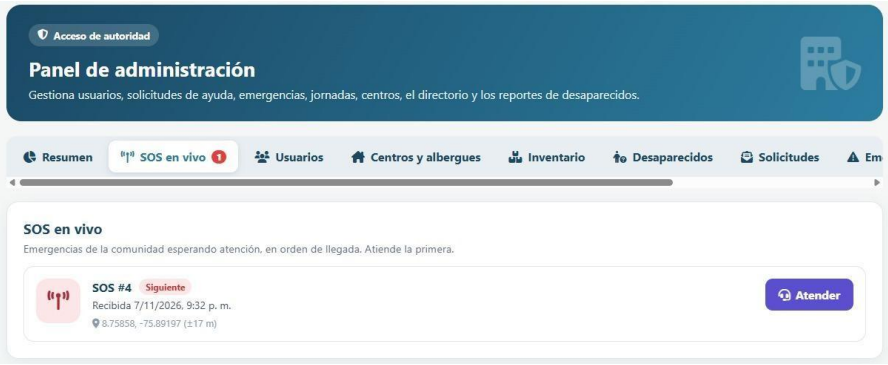


Figura 6. Cola de emergencias del operador con el SOS #4 en espera.

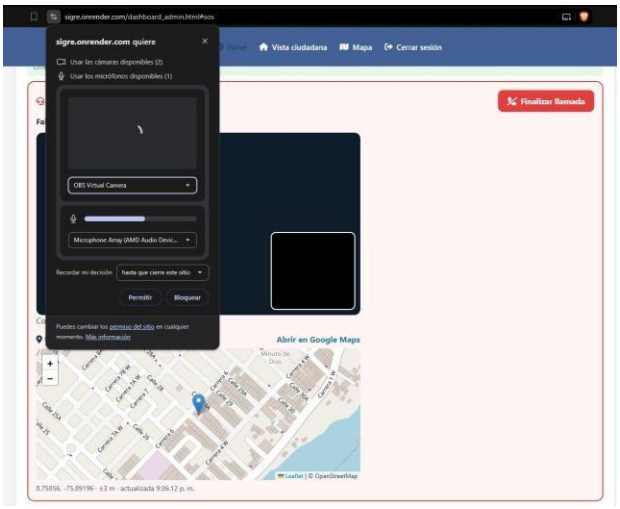


Figura 7. Permiso de cámara y micrófono al atender.

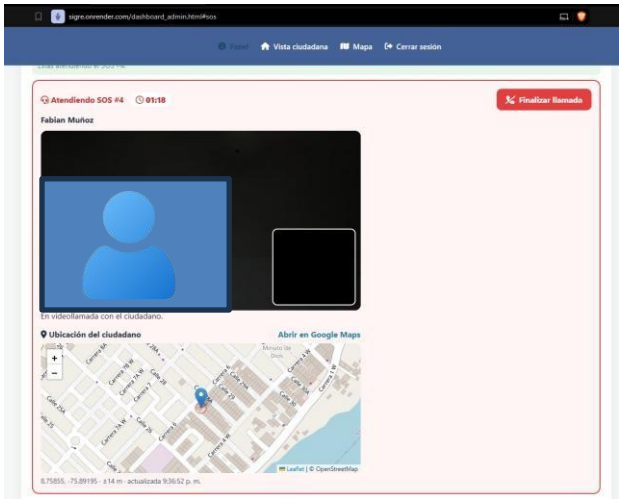
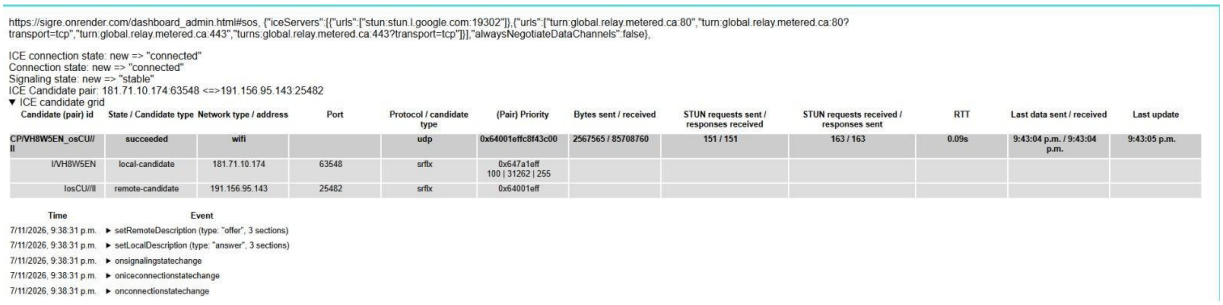


Figura 8. Panel en videollamada: video, cronómetro y mapa.



https://sigre.onrender.com/dashboard_admin.html#sos, {"iceServers":[{"urls":["stun:stun.l.google.com:19302"]},{"urls":["turn:global.relay.metered.ca:80","turn:global.relay.metered.ca:80?transport=tcp","turn:global.relay.metered.ca:443","turns:global.relay.metered.ca:443?transport=tcp"]}],"alwaysNegotiateDataChannels":false},

ICE connection state: new => "connected"
Connection state: new => "connected"
Signaling state: new => "stable"
ICE Candidate pair: 181.71.10.174:63548 <=>191.156.95.143:25482
▼ ICE candidate grid

Candidate (pair) id	State / Candidate type	Network type / address	Port	Protocol / candidate type	(Pair) Priority	Bytes sent / received	STUN requests sent / responses received	STUN requests received / responses sent	RTT	Last data sent / received	Last update
CP/VH8W5EN_osCU//ll	succeeded	wifi		udp	0x64001effc8f43c00	2567565 / 85708760	151 / 151	163 / 163	0.09s	9:43:04 p.m. / 9:43:04 p.m.	9:43:05 p.m.
I/VH8W5EN	local-candidate	181.71.10.174	63548	srflx	0x647a1eff 100 | 31262 | 255						
losCU//ll	remote-candidate	191.156.95.143	25482	srflx	0x64001eff						

Time	Event
7/11/2026, 9:38:31 p.m.	▶ setRemoteDescription (type: "offer", 3 sections)
7/11/2026, 9:38:31 p.m.	▶ setLocalDescription (type: "answer", 3 sections)
7/11/2026, 9:38:31 p.m.	▶ onsignalingstatechange
7/11/2026, 9:38:31 p.m.	▶ oniceconnectionstatechange
7/11/2026, 9:38:31 p.m.	▶ onconnectionstatechange

Figura 9. chrome://webrtc-internals: par de candidatos succeeded (srflx) y servidores ICE.

Prueba 5: tres emergencias en cola

Cómo se probó: script pruebas/probar_cola_sos.py. Tres ciudadanos piden SOS y luego un operador atiende la primera.
Resultado esperado: posiciones 1, 2 y 3; al atender la primera, las restantes pasan a 1 y 2.

Resultado obtenido: se obtuvieron las posiciones [1, 2, 3] y después [1, 2]. Los operadores recibieron las emergencias en menos de 0,3 s.

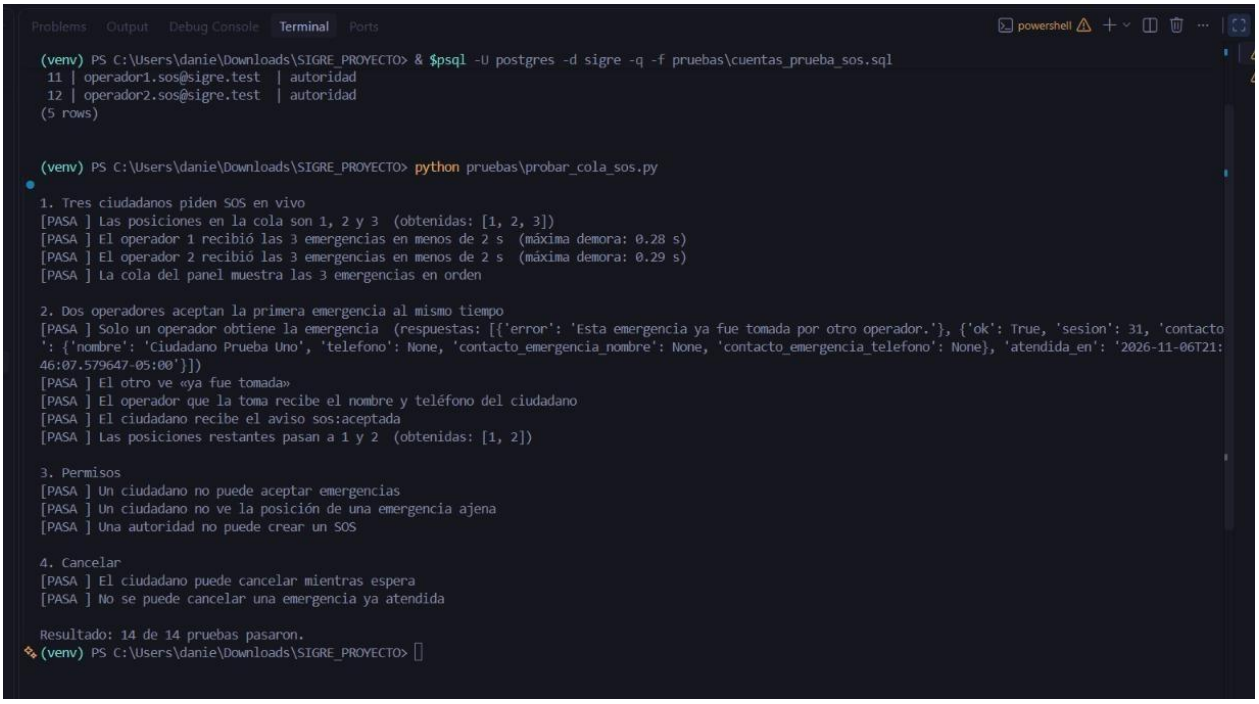


Figura 10. Resultado de probar_cola_sos.py (secciones 1 y 2).

Prueba 6: consulta directa a la base como ciudadano

Cómo se probó: primero, como postgres, se cuentan todas las sesiones SOS por ciudadano. Luego, con el usuario de la aplicación (sigre_app) y el script pruebas/probar_rls_sos.sql, se consulta la base identificándose como el ciudadano 8.
Resultado esperado: sin identificarse no ve ninguna sesión; como ciudadano 8 solo ve las suyas y ninguna ubicación ajena.

Resultado obtenido: la base tiene 32 sesiones de 4 ciudadanos; sin identificarse se ven 0; como ciudadano 8 se ven sus 9 sesiones, con 0 sesiones ajenas y 0 ubicaciones ajenas (seguridad por filas).

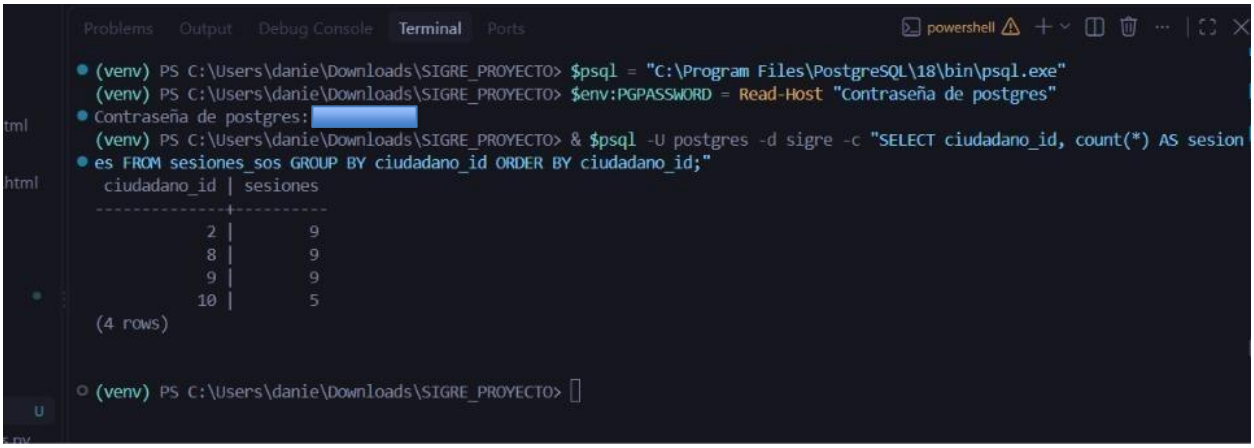


Figura 11. Todas las sesiones, vistas como postgres.

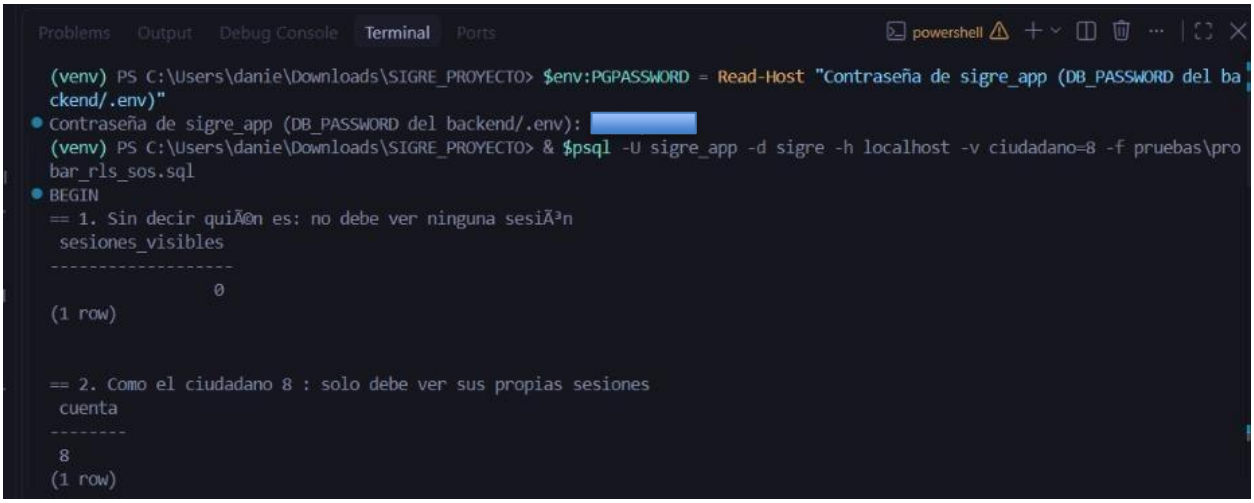


Figura 12. Consulta como ciudadano 8 (secciones 1 y 2).

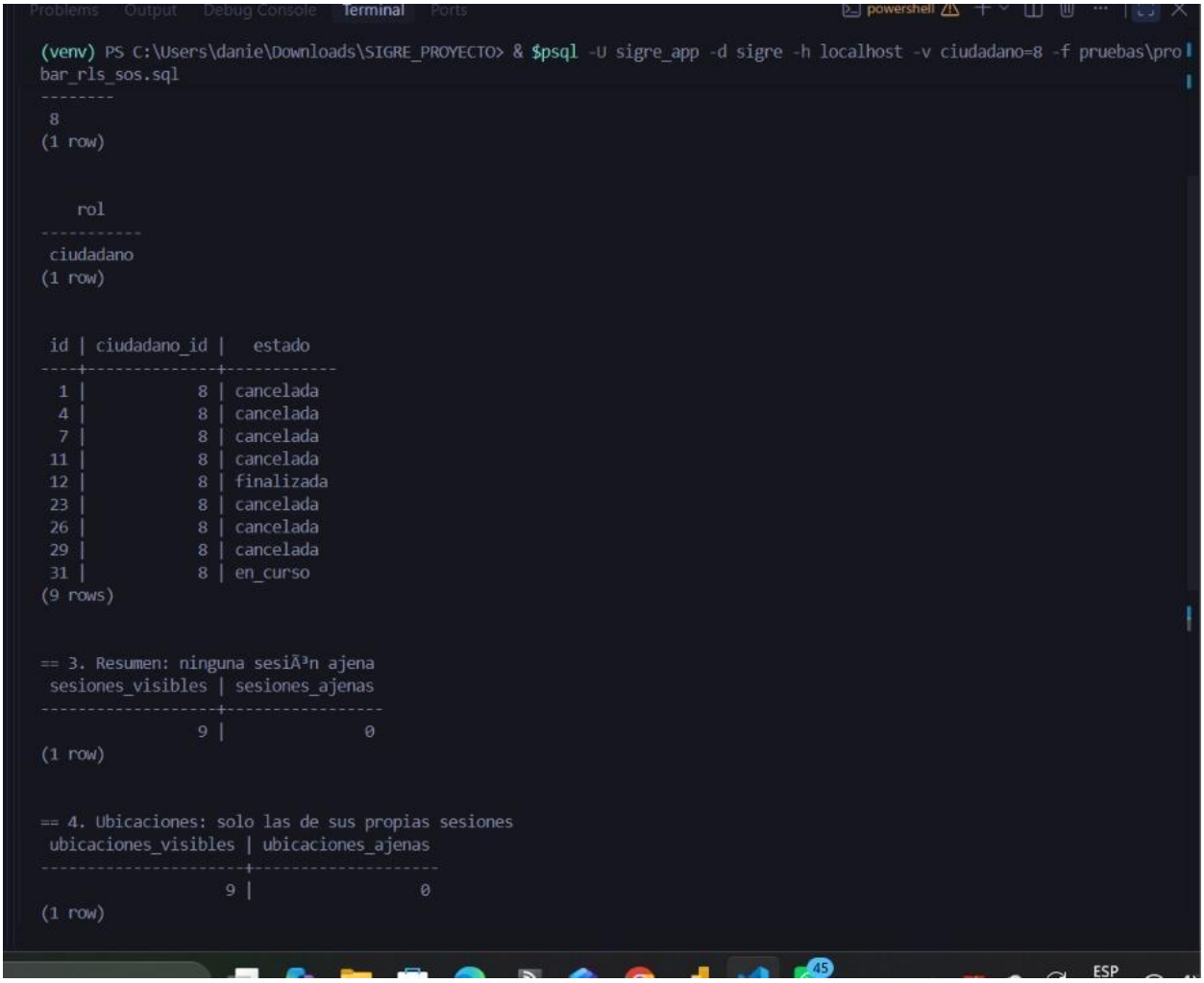


Figura 13. Consulta como ciudadano 8 (secciones 2 a 4).